

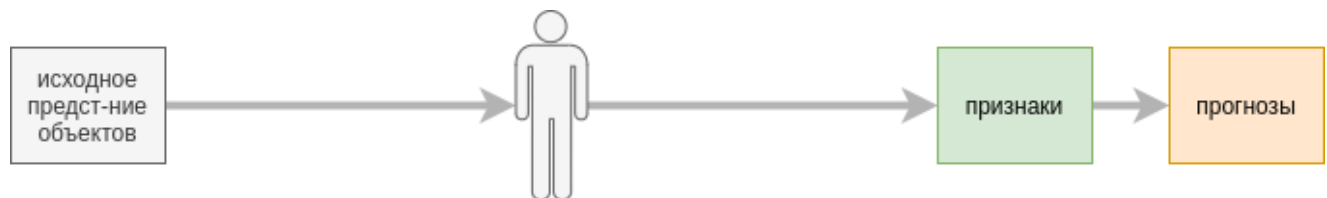
| 2026-03-18

| Глубокое обучение и искусственные нейронные сети

Глубокое (глубинное) обучение (Deep Learning) — разновидность машинного обучения, использующая для решения задач машинного обучения комплексные модели, построенные на принципе искусственных нейронных сетей.

Основным отличием от классических моделей машинного обучения («неглубокого» машинного обучения, shallow learning) является то, что в процессе глубокого обучения формируются не только результаты на выходе модели, но также и последовательность преобразований для получения существенных для решения задачи признаков из сырых данных.

Классическое машинное обучение полагается на человека для выявления таких признаков:



Глубокое обучение **самостоятельно формирует цепочку представлений**, приводящих к признакам:



Построение полезных промежуточных признаковых описаний объектов называется **обучением представлений (representation learning)**.

| Искусственный нейрон

Искусственный нейрон — математическая модель биологической клетки (естественного нейрона), которая повторяет следующие основные принципы:

1. нейрон связан с другими нейронами
2. каждая такая связь может быть сильнее или слабее, у нее есть **вес**
3. нейрон переходит в активное состояние в зависимости от суммы его входных сигналов

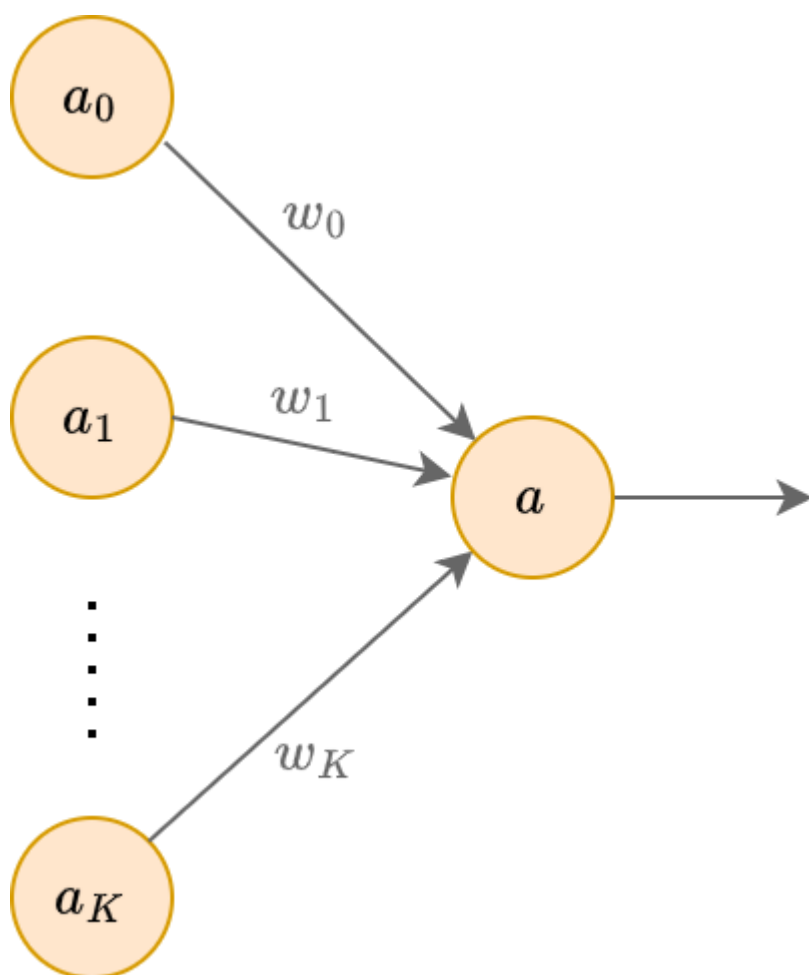
Аналитически это записывается так:

$$a = h \left(\sum_{k=0}^K w_k a_k \right),$$

где a_k — входные сигналы (или признаки x_k), w_k — веса связей, w_0 (или b , bias) — смещение, а $h(\cdot)$ — функция активации, определяющая правило перехода в активное состояние

Если более детально:

- a — выход нейрона (сигнал, который он посылает другим нейронам)
- $a_1 \dots a_k$ — нейроны, связанные с текущим нейроном входящей связью
- $w_0, w_1 \dots w_k$ — веса связей, которые определяют силу сигнала от k -того нейрона к текущему нейрону a



Задачи, решаемые искусственным нейроном

Одиночный искусственный нейрон может решать задачу регрессии и задачу классификации применительно к N -мерному вектору признаков $\mathbf{x} = [x^1, x^2, \dots x^N]$.

Для этого необходимо определить набор входных нейронов:

$$a_0 \equiv 1, a_1 = x^1, a_2 = x^2, \dots, a_K = x^N$$

Применяя этот набор входных нейронов к одиночному нейрону a , мы получаем закономерность:

$$y(\mathbf{x}) = h \left(w_0 + \sum_{i=0}^N w_i x^i \right)$$

w_0 — смещение, которое также можно обозначить как b (bias). Все параметры $w_0, w_1, \dots, w_N$ настраиваются по данным.

Конкретную задачу (регрессии или классификации) мы выбираем путем выбора функции активации.

I Регрессия

Для решения нейроном задачи регрессии выбирают **тождественную функцию активации** $h(u) = u$

$$y(\mathbf{x}) = w_0 + \sum_{i=0}^N w_i x^i$$

I Бинарная классификация

Для решения нейроном задачи классификации на два класса выбирают **сигмоидную функцию активации** $h(u) = \sigma(u) = \frac{1}{1+e^{-u}}$

Это делают потому, что сигмоида принимает значения на интервале $(0, 1)$, что можно использовать для отнесения значений, близких к нулю, к одному классу, а значений, близких к единице, к другому классу.

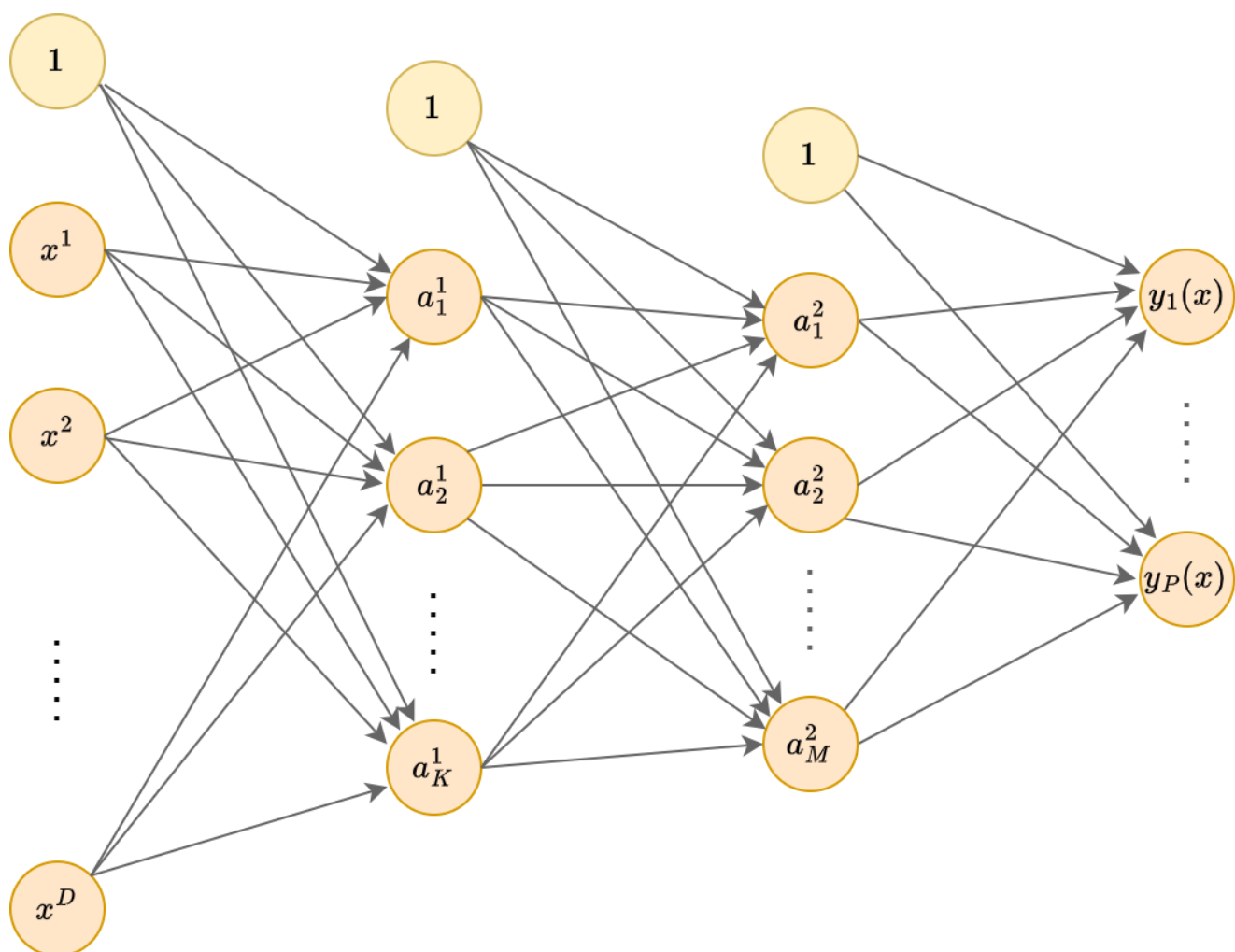
Другими словами, ее значение будет трактоваться как вероятность «положительного класса» ($p(y = +1|\mathbf{x})$), а вероятность противоположного класса можно найти через $p(y = -1|\mathbf{x}) = 1 - p(y = +1|\mathbf{x})$.

Эта же функция реализует бинарную логистическую регрессию.

I Многослойный перцептрон (MLP, multilayer perceptron)

Один нейрон может решать только простые задачи (регрессия или бинарная логистическая регрессия).

Для поиска сложных нелинейных зависимостей и решения сопутствующих задач применяется полносвязная искусственная нейронная сеть — **многослойный перцептрон (MLP)**



1. **Входной слой (input layer)**: вектор признаков объекта x .
2. **Скрытые слои (hidden layers)**: один или несколько слоев. Они строят **промежуточное представление признаков (intermediate feature representation)**. Если данных много, промежуточные признаки окажутся более информативными, чем заданные вручную
3. **Выходной слой (output layer)**: выдает финальное предсказание. Он может быть с 1 или более нейронами (зависит от задачи и используемого подхода к трактовке результатов).

Перцептрон обозначается по числу своих скрытых слоев + выходной слой; входной слой никогда не учитывается при описании структуры сети. На рисунке выше приведен **трехслойный** перцептрон.

Слои называются **полносвязными (fully connected layers, FCL)**, если между нейронами соседних слоев используется принцип связи «каждый с каждым»

Преобразования, выполняемые сетью над входным вектором x , могут быть представлены следующим образом:

$$\hat{y} = f(x) = g(W_3 h(W_2 h(W_1 x + b_1) + b_2) + b_3),$$

где:

- $W_1 \in \mathbb{R}^{K \times D}, b_1 \in \mathbb{R}^K$ — матрица весов и вектор смещений для 1-го слоя;
- $W_2 \in \mathbb{R}^{M \times K}, b_2 \in \mathbb{R}^M$ — матрица весов и вектор смещений для 2-го слоя;
- $W_3 \in \mathbb{R}^{P \times M}, b_3 \in \mathbb{R}^P$ — матрица весов и вектор смещений для 3-го слоя;
- $h()$ — функция активации скрытого слоя (допустимо также, что функция отличается от слоя к слою)
- $g()$ — функция активации (нелинейности) выходного слоя.

Обобщая:

1. W_i — матрицы весов
2. b_i — векторы смещений
3. $h(\cdot)$ — функция активации скрытых слоев
4. $g(\cdot)$ — функция активации выходного слоя

Tip

Скрытые слои строят, в соответствии с принципом глубокого обучения, промежуточное представление признаков (intermediate feature representation) объекта, а непосредственно результат определяется по извлеченным признакам идущего перед выходным скрытого слоя.

Промежуточные признаки строятся автоматически за счет подбора весов в процессе обучения. Если данных достаточно много, они достаточно разнообразные, то такие представления могут оказаться более полными, чем те, которые мог бы реализовать человек-аналитик.

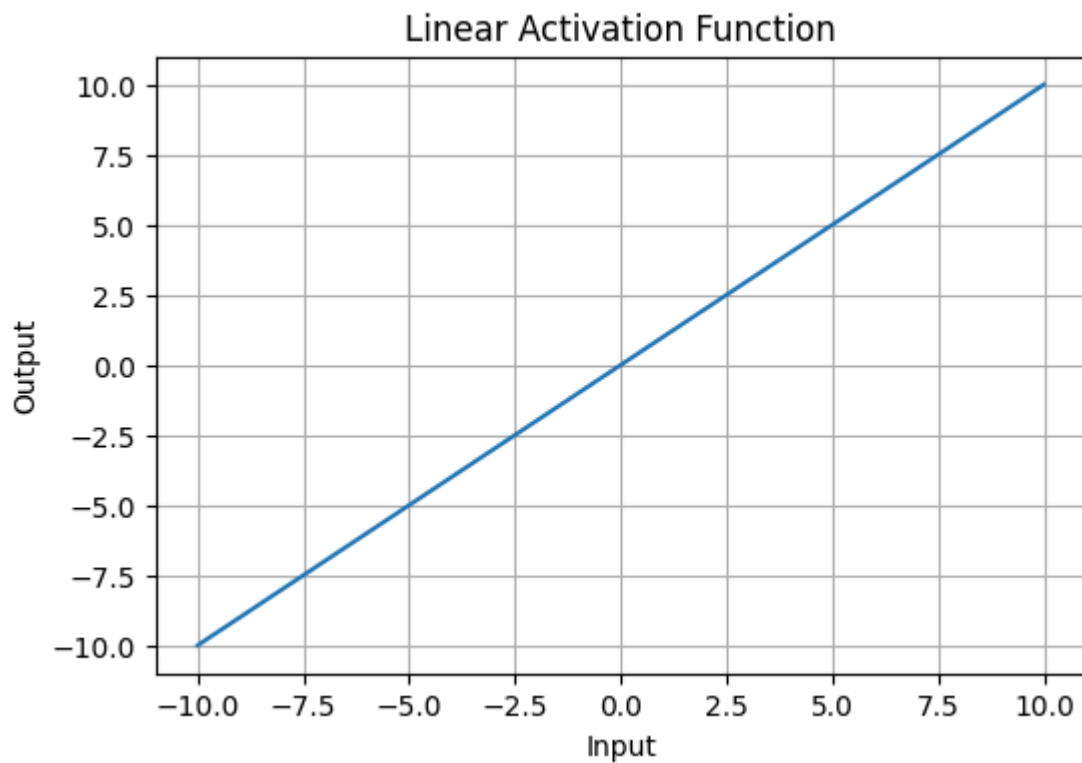
В каждом слое используется нейрон с константным значением (1) для моделирования смещения при расчете взвешенной суммы активации входных нейронов. Этот нейрон не принимает никаких входных сигналов, только отдает единицу.

Это дает на первом слое $(D + 1) \times K$ параметров, на втором — $(K + 1) \times M$, на третьем — $(M + 1) \times P$.

| Функции активации искусственных нейронов

| Линейная функция активации (тождественная, linear/identity)

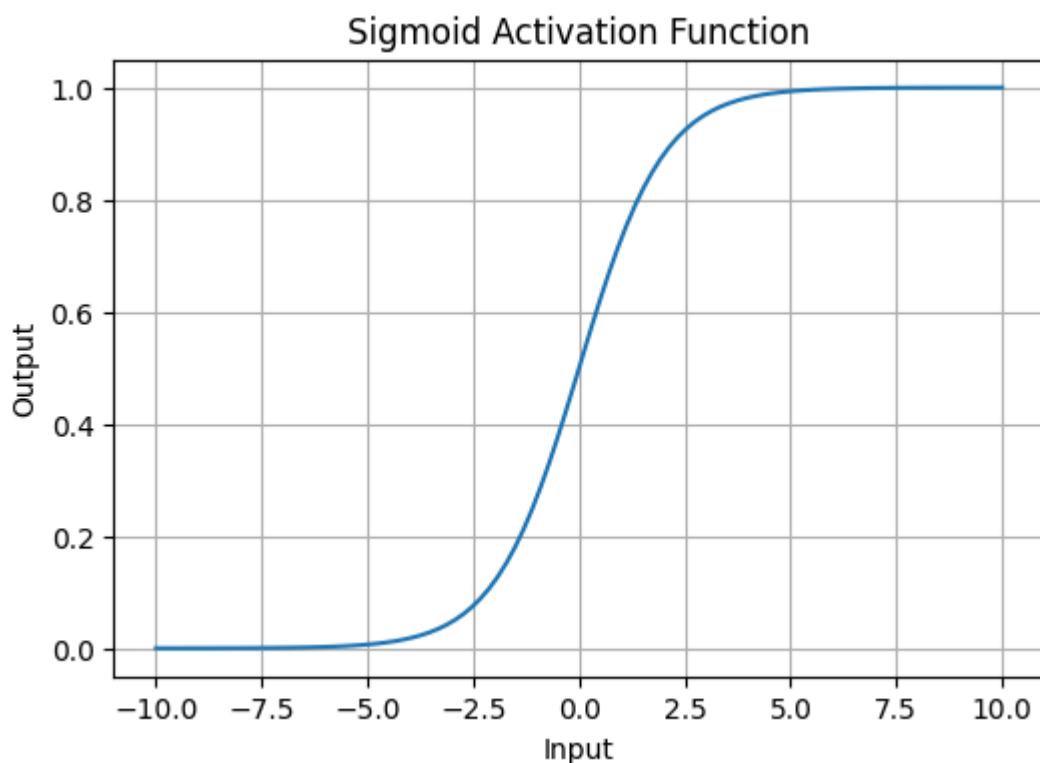
$$h(u) = u$$



Применяется в выходных слоях для моделирования регрессионного выхода. В скрытых слоях использование КРАЙНЕ ограничено, т.е. суперпозиция линейных функций вновь приводит к линейной функции.

Сигмоидная функция (сигмоида, sigmoid)

$$h(u) = \sigma(u) = \frac{1}{1 + e^{-u}}$$

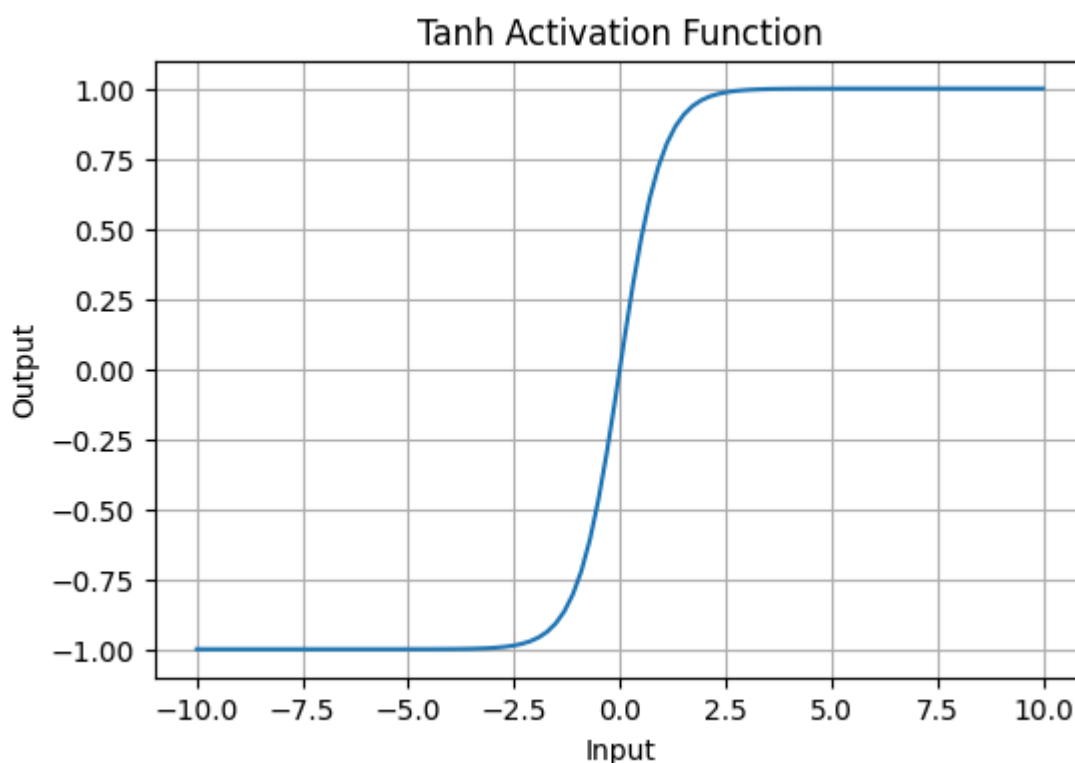


Принимает значения в диапазоне $(0, 1)$. Может использоваться в выходных слоях для решения задачи бинарной классификации (вероятность положительного класса).

В скрытых слоях не используется, т.к. за пределами $(-3, 3)$ быстро выходит на асимптоты -1 и $+1$ и почти не меняется, т.е. градиент близок к нулю, что может критически замедлить обучение сети или даже застопорить активации слоев с сигмоидой в районе значения -1 или 1 (проблема **затухания градиентов (vanishing gradient)**, из-за чего сеть перестает обучаться).

Гиперболический тангенс (tanh)

$$h(u) = \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} = 2\sigma(2u) - 1$$



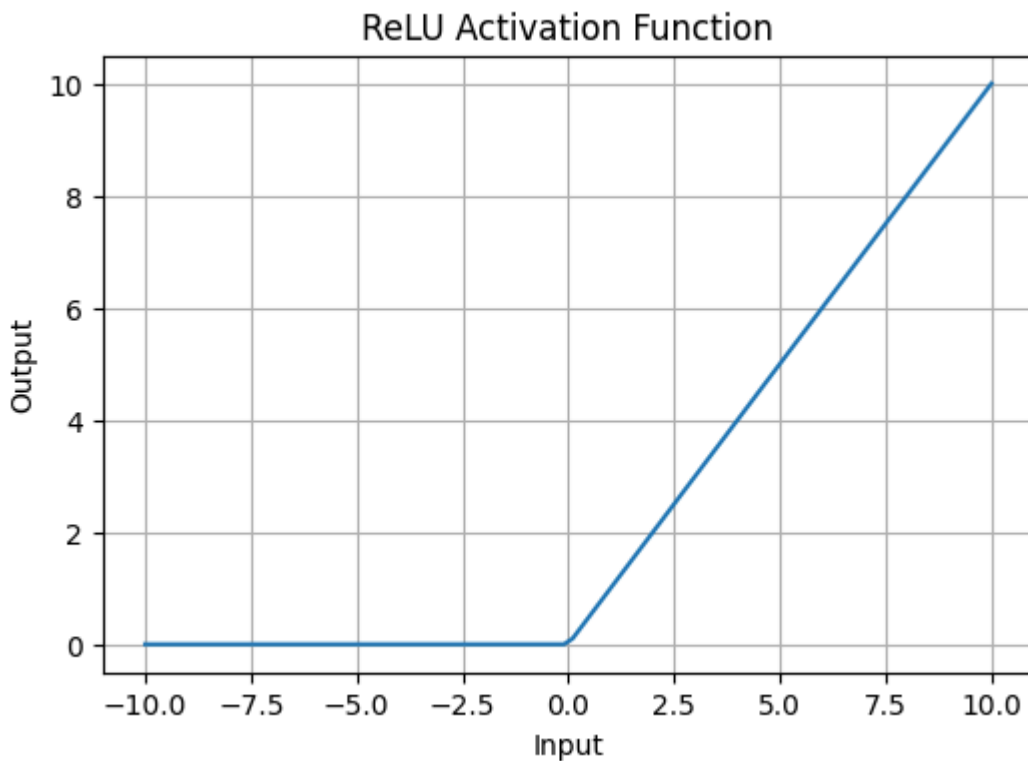
С точностью до линейного сжатия и сдвига совпадает с сигмоидой (и может быть выражен через нее, но, в отличие от сигмоиды, является нечетной функцией (т.е. для него справедливо $\tanh(-u) = -\tanh(u)$).

Принимает значения $(-1, 1)$ и центрирована по нулю. Обладает тем же недостатком, что и сигмоида — за интервалом $(-3, 3)$ быстро выходит на горизонтальные асимптоты с близким к нулю градиентом.

Используется преимущественно в выходных слоях регрессии с ограничением выхода (например, $[-1, 1]$).

Rectified linear unit (ReLU, спрямленный линейный блок)

$$h(u) = \max(0, u)$$



Обладает постоянным градиентом $+1$ на полуоси $u > 0$, что позволяет ее использовать как на скрытых слоях, так и на выходных.

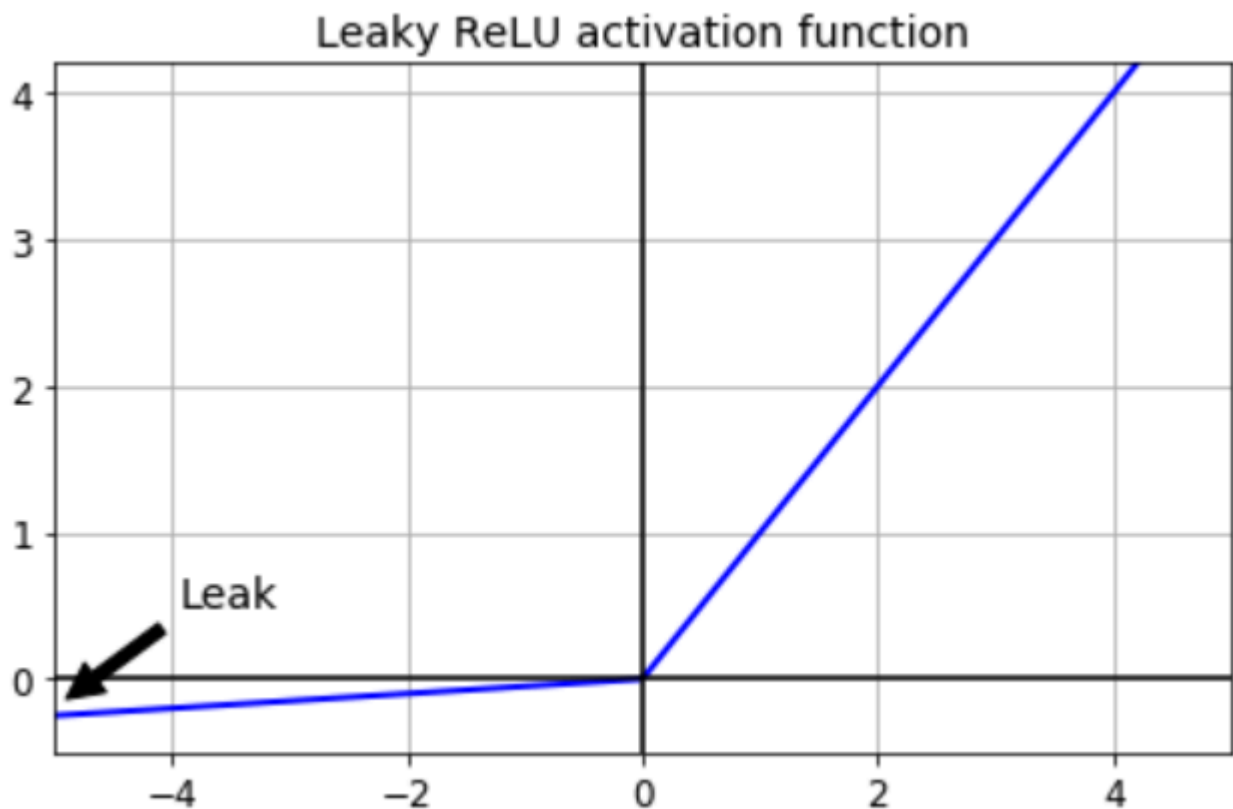
Еще одним преимуществом является то, что она гораздо устойчивее при квантизации моделей, т.е. ее вычислять быстрее (т.к. задействуются более простые операции для ее вычисления), а результат будет более стабильным при низкобитных представлениях чисел в облегченных сетях (например, Q4).

В совокупности все это делает ReLU одной из наиболее популярных функций активации.

Минусом функции является то, что любые отрицательные значения зануляются, что приводит к лишним вычислениям для тех нейронов, которые получают только отрицательные значения — такой нейрон все равно будет тратить ресурсы, но не сможет никак повлиять на веса сети, тем самым становясь бесполезным (или «мертвым» — проблема «dead neuron»).

Leaky ReLU

$$h(u) = \max(\alpha u; u), \alpha \in (0, 1)$$



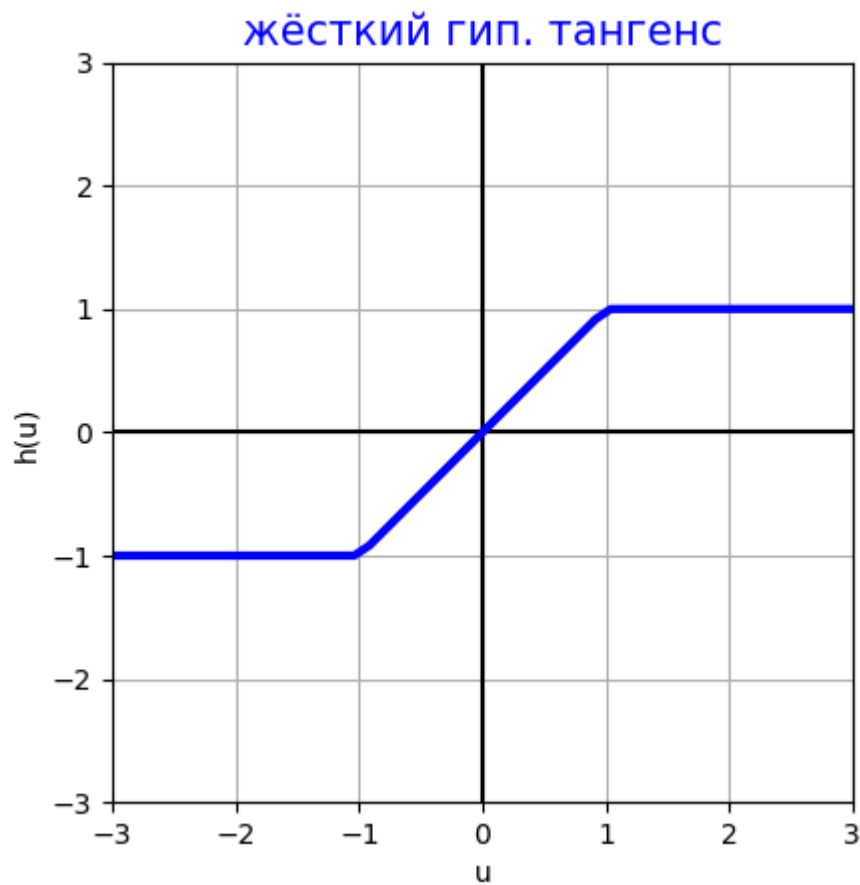
Призвана решить проблему «мертвых» нейронов при использовании обычной ReLU за счет добавления гиперпараметра α , который обеспечивает градиент в отрицательной области, что позволяет нейрону выдавать нетривиальные значения всегда, не вырождаясь в 0.

Считается одним из наиболее сильных вариантов для использования в актуальных нейронных сетях. Рекомендуется как отличный базовый вариант для скрытых слоев.

Обычно в реализациях Leaky ReLU $\alpha = 0.01$, но рекомендуется настраивать данный гиперпараметр в зависимости от используемых архитектур и задач.

Жесткий гиперболический тангенс (hard tanh)

$$h(u) = \max(-1; \min(u; +1))$$

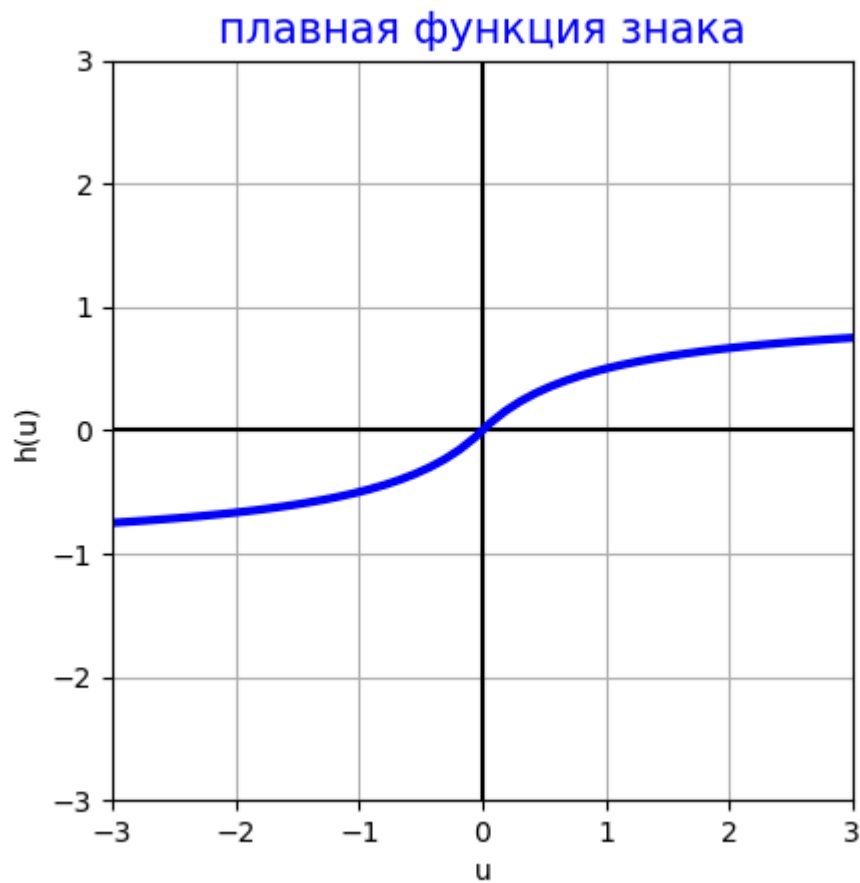


Область применения такая же, как и у обычного гиперболического тангенса, но вычисляется `hard tanh` гораздо быстрее за счет использования более простых операций.

Повторяет одно из ключевых преимуществ ReLU — остается точным и устойчивым при низкобитных представлениях чисел при квантовании сетей.

Плавная знаковая функция (SoftSign)

$$h(u) = \frac{u}{(1 + |u|)}$$



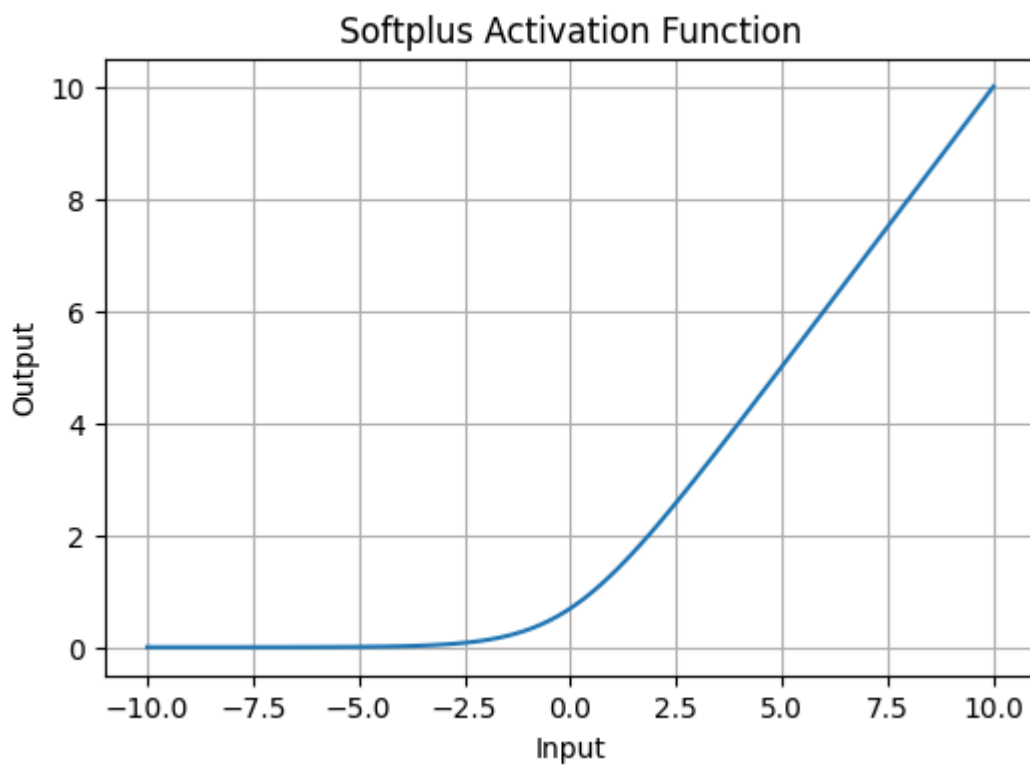
Концептуально повторяет гиперболический тангенс, но медленнее выходит на константы, что улучшает сходимость сети в процессе обучения.

Еще одно преимущество — проще вычисляется по сравнению с гиперболическим тангенсом.

Недостаток тот же, что и у $\tanh$: из-за вырождения в константы -1 , $+1$ использование в скрытых слоях ограничено.

| Неотрицательная знаковая функция (SoftPlus)

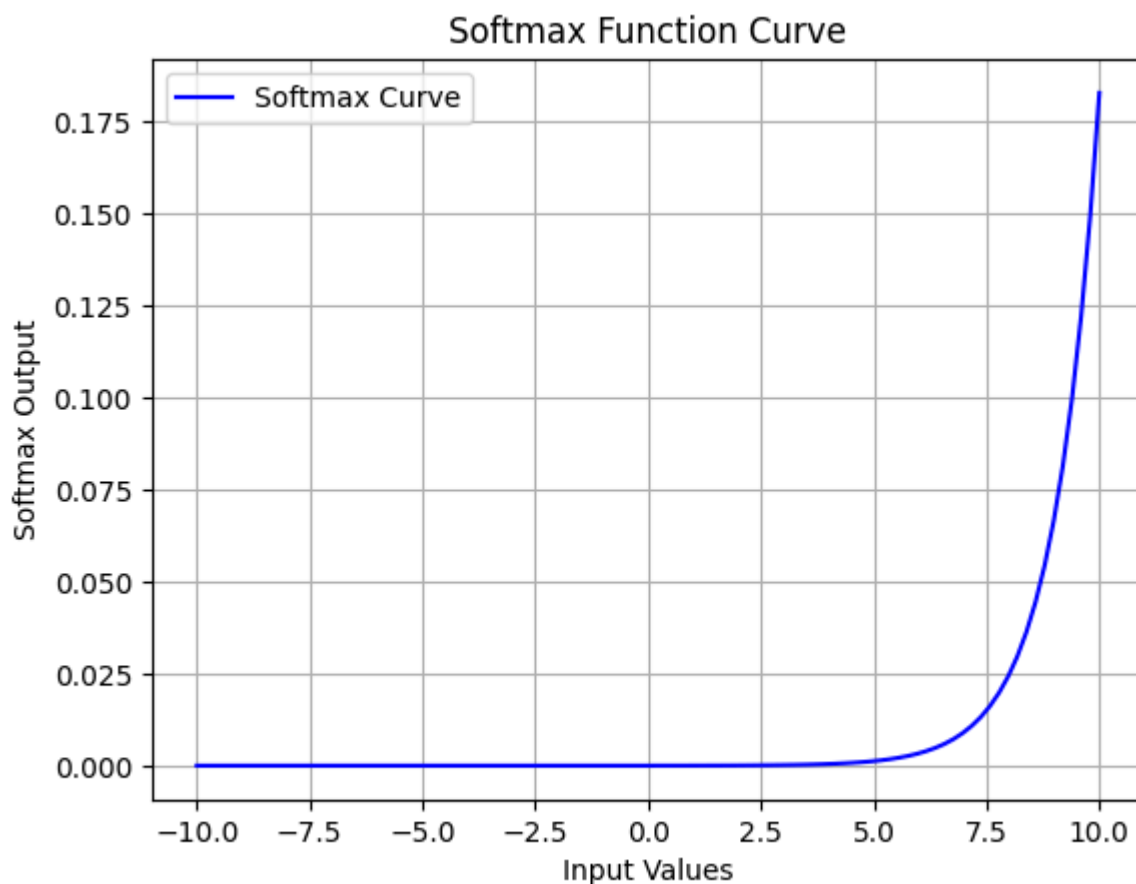
$$h(u) = \ln(1 + e^u)$$



Имеет градиент уже на полуоси $u \geq -3$, что позволяет использовать ее в скрытых слоях. В выходных слоях применяется в задачах, где нужно неотрицательное значение.

SoftMax

$$h(u_i) = \frac{e^{u_i}}{\sum_{j=1}^K e^{u_j}}$$



Преобразует выходы сети в вероятности путем нормализации выходных значений каждого класса до диапазона $(0, 1)$, при этом гарантируя, что сумма вероятностей остается равной 1.

Применяется в выходных слоях для классификации с множеством классов (многоклассовой классификации).

| Процедура обучения нейронных сетей

Обучение нейронной сети — подбор (оптимизация) матриц весов W и смещений b , чтобы минимизировать функцию потерь (Loss).

| Функция потерь (Loss Function)

Функция потерь **оценивает ошибку предсказания**

Часто используются:

- **MSE (Mean Squared Error)**: для регрессии.
- **Cross-Entropy Loss (Кросс-энтропия)**: для многоклассовой классификации (использует метод максимального правдоподобия)

| Метод стохастического градиентного спуска (SGD)

Обычный градиентный спуск требует вычисления градиента по всей выборке, что долго.

SGD использует быстро вычисляемую аппроксимацию — градиент считается на случайной выборке малого размера, называемой **мини-батчем (minibatch)**

На практике данные разбиваются на мини-батчи (например, 64 картинки), и веса обновляются после каждого прохода батча. Полный проход по всем данным называется **эпохой (epoch)**

$$W_{new} = W_{old} - \eta \cdot \nabla L,$$

где η — шаг обучения (learning rate)

В современных сетях часто применяют адаптивные алгоритмы поверх SGD, например, **Adam / AdamW**, которые подбирают шаг индивидуально для каждого веса.

| Обратное распространение ошибки (Backpropagation)

Для вычисления градиента ∇L по каждому из миллионов весов сети используется метод **Backpropagation (backward-mode automatic differentiation)**

Алгоритм состоит из двух шагов:

1. **Проход вперед (Forward pass)**: слева направо вычисляются и запоминаются значения всех промежуточных узлов (активации слоев).
2. **Проход назад (Backward pass)**: справа налево, используя **правило цепочки (chain rule)** из математического анализа, рекуррентно вычисляются производные функции потерь по каждому весу

Tip

Библиотеки глубокого обучения (например, PyTorch) строят динамический граф вычислений и делают это автоматически (модуль Autograd), что избавляет от необходимости ручного вычисления производных

| PyTorch

В исследовательском сообществе и инженерии **PyTorch** в настоящее время стал стандартом

Ключевые концепции:

1. **Tensor (Тензор)**: аналог массивов NumPy, но с поддержкой вычислений на видеокартах (GPU) и автоматического дифференцирования.
2. **Autograd**: модуль, который сам считает производные для Backpropagation.

3. `torch.nn.Module` : базовый класс для всех нейросетей. Позволяет легко комбинировать слои (`nn.Linear` , `nn.ReLU`).
4. `DataLoader` : удобный класс для загрузки датасетов мини-батчами.

| Лабораторная работа №4

Исходные данные — датасет MNIST.

- Включает 60,000 обучающих и 10,000 тестовых черно-белых изображений рукописных цифр (от 0 до 9).
 - Размер каждого изображения: 28x28 пикселей.
 - Загружается через встроенный модуль `torchvision.datasets` .
1. Используя `torchvision.datasets.MNIST` , скачайте обучающую и тестовую выборки.
 2. Примените трансформацию в тензор `torchvision.transforms.ToTensor()` .
 3. Создайте `DataLoader` для обучающей и тестовой выборок с размером батча (`batch_size = 64` или `128`).
 4. Выведите на экран форму (shape) первого батча изображений и меток. Визуализируйте 3-5 случайных изображений с помощью `matplotlib` .
 5. Реализуйте класс нейронной сети, унаследовав его от `nn.Module` .
 1. **Входной слой**: изображение 28x28 вытяните в одномерный вектор размером 784 (`nn.Flatten()`).
 2. **Скрытые слои**: добавьте 2 полносвязных слоя (`nn.Linear`), например, 784 → 256 нейронов → 128 нейронов.
 3. **Функции активации**: после каждого скрытого слоя применяйте `nn.ReLU()` или `nn.LeakyReLU()` .
 4. **Выходной слой**: полносвязный слой, переводящий размерность в 10 классов (`nn.Linear(128, 10)`).
 6. Инициализируйте полученную модель.
 7. Определите функцию потерь: `criterion = nn.CrossEntropyLoss()` (она внутри себя вычислит и функцию Softmax, поэтому отдельно ее применять не надо).
 8. Определите оптимизатор: `optimizer = torch.optim.AdamW(model.parameters(), lr=0.001)` .
 9. Реализуйте цикл из 5-10 эпох обучения (можно на GPU, если есть возможность).
Для каждой эпохи:
 1. `model.train()`
 2. Запускайте цикл по батчам:
 1. Прямой проход (forward pass).
 2. Вычисление loss.
 3. `optimizer.zero_grad()`
 4. `loss.backward()`
 5. `optimizer.step()`

3. Сохраняйте среднее значение потерь за эпоху

10. Проведите оценку модели

1. `model.eval()`

2. `with torch.no_grad(): ...`

3. Пройдите по тестовому `DataLoader` и посчитайте Accuracy (должна быть > 97%).

11. Анализ результатов

1. Постройте график функции потерь (Loss) от эпохи.

2. Визуализируйте предсказания для нескольких тестовых картинок